

Jenkins CI/CD Training Notes

Comprehensive notes covering Jenkins fundamentals, CI/CD concepts, DevOps basics, AI prompting, AWS labs, and Jenkins installation.

Table of Contents

- Introduction
 - What is CI and CD?
 - DevOps Overview
 - Jenkins Overview
 - History of Jenkins
 - Key Features of Jenkins
 - Cloud Service Models
 - AI & Prompt Engineering Basics
 - LLMs vs Foundation Models
 - Context Window
 - Hands-on Labs
 - Useful Linux Commands
-

Introduction

Jenkins is one of the most popular open-source automation servers used for implementing **Continuous Integration (CI)** and **Continuous Delivery/Deployment (CD)**.

During this training you will learn:

- Jenkins fundamentals
- Jenkins UI
- CI/CD Pipelines
- AWS Integration
- Git Integration
- Docker Integration
- Plugin Management
- DevOps Concepts

- Basic AI Prompt Engineering for DevOps
-

Jenkins Fundamentals

Topics covered:

- Jenkins Installation
 - Jenkins Dashboard
 - Jobs
 - Builds
 - Source Code Management (SCM)
 - Plugins
 - Build History
 - Pipeline
 - Testing Integration
 - Deployment Automation
-

What is CI (Continuous Integration)?

Continuous Integration is the practice of automatically integrating code changes from multiple developers into a shared repository.

Typical developer workflow:

```
Developer
↓
Write Code
↓
Commit Code
↓
Push to Git Repository
↓
Pull Request
↓
Code Review
↓
Merge to Main Branch
↓
Automated Jenkins Pipeline
↓
Build
↓
Testing
```

Example

A development team may commit code **20+ times a day**.

Instead of manually building and testing every change, Jenkins automatically:

- Detects the new commit
- Pulls latest code
- Builds the application
- Runs automated tests
- Reports success/failure

This is Continuous Integration.

What is CD (Continuous Delivery / Continuous Deployment)?

Once CI is complete and all tests pass, the application is ready for deployment.

```
Source Code
  ↓
Build
  ↓
Testing
  ↓
Artifact Ready
  ↓
Deploy to Development
  ↓
Deploy to Staging
  ↓
Deploy to Production
```

There are two common CD approaches:

Continuous Delivery

Deployment is automated **up to production**, but requires **human approval** before releasing.

```
Build
  ↓
Test
  ↓
Ready
  ↓
Manual Approval
  ↓
Production
```

Continuous Deployment

Everything is automated.

```
Build
  ↓
```

```
Test
↓
Deploy
↓
Production
```

No manual intervention is required.

DevOps Overview

DevOps is a culture that improves collaboration between Development and Operations teams.

Goals:

- Faster software delivery
- Early testing
- Fail fast
- Automated deployments
- Continuous monitoring

Popular DevOps tools:

- Git
 - Jenkins
 - Docker
 - Kubernetes
 - Terraform
 - Ansible
-

Jenkins in DevOps

Examples:

AWS Automation

Write a Jenkins Pipeline that:

- Creates EC2 instances
- Configures them
- Destroys them automatically

```
Jenkins
↓
AWS Plugin
↓
```

Docker Automation

Pipeline can:

- Build Docker Images
- Tag Images
- Push Images to Docker Hub

```
Git
↓
Jenkins
↓
Docker Build
↓
Docker Hub
```

Jenkins Overview

Jenkins is:

- Open-source
- Written in Java
- Automation Server
- Extensible using plugins

Primary use cases:

- Continuous Integration
- Continuous Delivery
- Build Automation
- Testing Automation
- Deployment Automation

It automates software delivery in a reliable and repeatable manner.

History of Jenkins

Year	Event
2004	Hudson created at Sun Microsystems
2010	Oracle acquired Sun Microsystems

Year	Event
2011	Hudson renamed to Jenkins
Today	One of the world's most popular CI/CD tools

Key Features of Jenkins

1. Pipeline Support

Pipelines define complete software workflows.

Typical stages:

```
Checkout
  ↓
Build
  ↓
Test
  ↓
Package
  ↓
Deploy
```

Pipeline code is stored inside a **Jenkinsfile**.

Language used:

- Groovy DSL (Domain Specific Language)

2. Plugin Ecosystem

Jenkins has thousands of plugins.

Examples:

- AWS
- Docker
- Kubernetes
- Terraform
- Git
- Maven
- SonarQube
- Slack

Plugins extend Jenkins functionality.

Restaurant Analogy

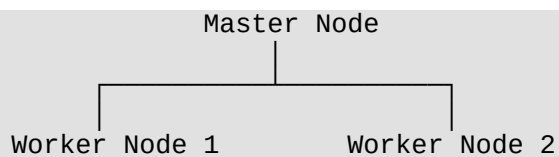
```
Customer
  ↓
Waiter (Jenkins)
  ↓
Kitchen
  ↓
Chef (Plugin)
  ↓
Food (Output)
```

Example:

AWS Plugin performs AWS operations and returns logs to Jenkins.

3. Distributed Builds

Jenkins supports multiple build agents.



Benefits:

- Better scalability
 - Faster builds
 - Parallel execution
-

4. Source Code Integration

Jenkins integrates with:

- Git
- GitHub
- GitLab
- Bitbucket
- Mercurial
- Subversion

Authentication methods:

- Username & Password
- Personal Access Token (PAT)
- SSH Keys

Environment Variables example:

```
$DOCKERHUB_USERNAME  
$AWS_ACCESS_KEY_ID  
$AWS_SECRET_ACCESS_KEY
```

5. Build Triggers

Time-based Trigger

Uses Cron expressions.

Examples:

- Every 5 minutes
 - Every 2 hours
 - Daily
 - Weekly
-

Webhook Trigger

Automatically starts a pipeline whenever code is pushed to GitHub.

```
GitHub Push  
  ↓  
Webhook  
  ↓  
Jenkins  
  ↓  
Pipeline
```

SCM Polling

Jenkins checks the repository periodically.

Example:

Every 10 minutes.

6. RBAC (Role-Based Access Control)

Manage user permissions.

Examples:

- Administrator
- Developer

- Viewer
 - Build Operator
-

Cloud Service Models

1. Infrastructure as a Service (IaaS)

Provides maximum infrastructure control.

Examples:

- Amazon EC2
- Azure Virtual Machines
- Google Compute Engine

Users choose:

- Operating System
- CPU
- RAM
- SSD/HDD
- IOPS

Suitable for:

- System Administrators
 - Infrastructure Teams
-

2. Platform as a Service (PaaS)

Developers focus on writing code.

Infrastructure is managed by the cloud provider.

Examples:

- AWS Elastic Beanstalk
- Amazon RDS
- Amazon Aurora

Databases:

- MySQL

- PostgreSQL
-

3. Function as a Service (FaaS)

Serverless computing.

Example:

AWS Lambda

Features:

- Auto Scaling
 - Pay-per-use
 - No server management
-

4. Software as a Service (SaaS)

Ready-to-use software over the internet.

Examples:

- ChatGPT
 - Gmail
 - Microsoft 365
-

AI & Prompt Engineering

Modern DevOps engineers increasingly use AI tools such as:

- ChatGPT
- Claude
- Kiro
- GitHub Copilot

Example use case:

"Create a Jenkins Pipeline that launches two EC2 instances in the us-east-1 region using t2.medium instances."

Prompt Engineering Techniques

Zero-shot Prompt

Ask directly.

Example:

What is the capital of Italy?

Few-shot Prompt

Provide examples before asking.

Example:

English → Spanish

Hello → Hola

Now translate:

Thank you

Role-based Prompting

Assign a role to the AI.

Example:

You are a DevOps Engineer.

Create a Jenkins Pipeline
that launches two EC2 instances
in us-east-1 using t2.medium.

LLM vs Foundation Models

Large Language Model (LLM)

Designed primarily for text generation.

Examples:

- GPT
- Claude
- Llama

Characteristics:

- Natural Language Processing
 - Billions of parameters
 - Text generation
-

Foundation Model (FM)

A broader category.

Can generate:

- Text
- Images
- Audio
- Video

LLMs are a subset of Foundation Models.

Single-modal vs Multi-modal Models

Single-modal

Input:

Text

Output:

Text

Multi-modal

Accepts multiple input types.

Examples:

- Text
- Images

Produces:

- Text
 - Images
 - Code
-

Popular AI Models

Company	Models
OpenAI	GPT-5.5, GPT-5.1, GPT-4.1
Anthropic	Claude Sonnet, Opus, Haiku
Meta	Llama
Amazon	Titan

Context Window

A context window is the amount of information an AI model can remember during a conversation.

Example:

```
Prompt 1
150 Tokens

↓

Response
2000 Tokens

↓

Prompt 2
↓

Entire conversation becomes context
```

Once the context limit is reached:

- Older information may be forgotten
- Start a new chat for a fresh context

Hands-on Labs

Lab 1 – Create an AWS Free Tier Account

Objective

Create an AWS Free Tier account that will be used throughout the Jenkins CI/CD training.

Prerequisites

- A valid email address
- A mobile phone number for verification
- A debit/credit card for identity verification (AWS may perform a temporary authorization; Free Tier eligible resources are available at no cost within limits.)

Step 1: Visit AWS

Open your browser and navigate to:

<https://aws.amazon.com/free/>

Click **Create a Free Account**.

Step 2: Create an AWS Account

Enter the following details:

- Email Address
- Password
- AWS Account Name

Example:

Email:
john.doe@gmail.com

AWS Account Name:
john-training

Click **Continue**.

Step 3: Verify Email Address

AWS sends a verification code to your email.

- Open your mailbox.
 - Copy the verification code.
 - Enter it on the AWS registration page.
-

Step 4: Create Root User Password

Choose a strong password that includes:

- Uppercase letters

- Lowercase letters
- Numbers
- Special characters

Example:

Example@123

Store the password securely.

Step 5: Contact Information

Select:

Personal Account

Fill in:

- Full Name
- Phone Number
- Country
- Address
- City
- State
- Postal Code

Accept the AWS Customer Agreement.

Step 6: Billing Information

Enter your debit or credit card details.

AWS uses this information for identity verification.

AWS Free Tier resources remain free as long as you stay within Free Tier limits.

Step 7: Identity Verification

Complete phone verification.

AWS sends an OTP to your registered mobile number.

Enter the OTP.

Step 8: Select Support Plan

Choose:

Basic Support (Free)

Click **Complete Sign Up**.

Step 9: Login to AWS Console

Visit:

<https://console.aws.amazon.com>

Sign in using your Root User credentials.

You should now see the AWS Management Console.

Lab Outcome

AWS Account Created

Successfully logged into AWS Console

Lab 2 – Create an IAM User

Objective

Create an IAM User with Administrator permissions instead of using the Root Account.

Best Practice: Never perform day-to-day activities using the Root User.

Step 1: Login as Root User

Login to the AWS Console using your Root Account.

Step 2: Open IAM

In the AWS search bar, type:

IAM

Open the **Identity and Access Management (IAM)** service.

Step 3: Navigate to Users

From the left navigation pane:

Users

Click:

Create User

Step 4: Configure User Details

Enter:

User Name :
your-name

Example:

rahul

Select:

Provide user access to the AWS Management Console

Choose:

I want to create an IAM user

Set a custom password or let AWS generate one.

Click **Next**.

Step 5: Assign Permissions

Choose:

Attach Policies Directly

Search for:

AdministratorAccess

Select:

AdministratorAccess

Click **Next**.

Step 6: Review and Create

Review all details.

Click:

Create User

Step 7: Download Login Details

Download or copy:

- IAM Username
- Console Login URL
- Password

These details will be required for future labs.

Step 8: Login Using IAM User

Sign out of the Root User.

Open the IAM login URL.

Login using:

- Username
- Password

Verify that you can access the AWS Console.

Lab Outcome

IAM User Created

AdministratorAccess Assigned

Successfully logged in using IAM User

Lab 3 – Launch an EC2 Instance

Objective

Launch an Ubuntu EC2 instance that will be used as the Jenkins Server.

Step 1: Open EC2 Dashboard

From the AWS Console, search for:

EC2

Open the EC2 Dashboard.

Step 2: Launch Instance

Click:

Launch Instance

Step 3: Configure Basic Details

Enter:

Setting	Value
Name	jenkins-server
AMI	Ubuntu Server 24.04 LTS
Architecture	64-bit (x86)

Step 4: Choose Instance Type

Select:

t3.micro

(Eligible under AWS Free Tier in many regions. If unavailable, use `t2.micro` where applicable.)

Step 5: Create a Key Pair

Click:

Create New Key Pair

Configure:

Name :

jenkins-key

Type:

RSA

Format:

.pem

Click:

Create Key Pair

The key pair will automatically download to your computer.

Important: Keep this .pem file safe. You will need it to SSH into the EC2 instance.

Step 6: Configure Network Settings

Keep the default VPC and subnet.

Create or use the default Security Group.

Allow the following inbound rule:

Type	Port
SSH	22

(Optional for later labs)

Type	Port
HTTP	80

Step 7: Configure Storage

Storage:

20 GB

Volume Type:

gp3

Leave the remaining settings as default.

Step 8: Review Configuration

Verify:

- Instance Name
- Ubuntu 24.04
- t3.micro
- Key Pair Selected

- SSH Enabled
 - 20 GB gp3 Storage
-

Step 9: Launch Instance

Click:

Launch Instance

AWS will provision the virtual machine.

Wait until the instance state changes to:

Running

Step 10: Verify Instance

Open the instance details.

Verify:

- Instance State = Running
- Status Checks = 2/2 Passed
- Public IPv4 Address is assigned

Copy the Public IP address.

It will be required in the next lab.

Understanding the EC2 Instance

Your EC2 server includes:

- Virtual CPU (vCPU)
- RAM
- Operating System
- Elastic Block Storage (EBS)
- Network Interface
- Security Group (Firewall)

Think of EC2 as a virtual computer running in the AWS cloud.

Lab Outcome

EC2 Instance Launched
Ubuntu Server Running
Public IP Address Available
Ready for SSH Connection

Lab 4 – SSH into EC2

Objective

Connect to the EC2 instance using SSH, verify the operating system, update the hostname, and check the Jenkins service status.

Step 1: Navigate to the Key Pair Directory

Open **Command Prompt** (Windows) or **Terminal** (Linux/macOS) and navigate to the directory where your downloaded key pair (.pem) file is stored.

```
cd Downloads
```

Replace the directory if your key pair is stored in a different location.

Step 2: Connect to the EC2 Instance

Use the following SSH command to connect to your Ubuntu EC2 instance.

```
ssh -i <key-pair.pem> ubuntu@<public-ip>
```

Replace:

- <key-pair.pem> with your downloaded key pair file.
- <public-ip> with the Public IPv4 address of your EC2 instance.

Example:

```
ssh -i jenkins-key.pem ubuntu@54.xxx.xxx.xxx
```

If prompted with:

```
Are you sure you want to continue connecting (yes/no)?
```

Type:

```
yes
```

Step 3: Verify the Logged-in User

Run the following command:

```
whoami
```

Expected output:

```
ubuntu
```

Step 4: Verify the Operating System

Check the Linux distribution installed on the EC2 instance.

```
cat /etc/os-release
```

This command displays information such as:

- Operating System Name
 - Version
 - Ubuntu Release
-

Step 5: Rename the Hostname

Set the hostname of the server to **jenkins**.

```
sudo hostnamectl set-hostname jenkins
```

Step 6: Reload the Shell

Reload the current shell session.

```
bash
```

Step 7: Verify Jenkins Service Status

Check whether the Jenkins service is installed and running.

```
sudo systemctl status jenkins
```

Lab Outcome

Successfully connected to the EC2 instance using SSH

Verified the logged-in user

Verified the operating system

Updated the server hostname to **jenkins**

Checked the Jenkins service status

Installing Nginx

Objective

Install the Nginx web server on the Ubuntu EC2 instance and verify that it is running successfully.

Step 1: Update Package Repository

Update the package index.

```
sudo apt update -y
```

Step 2: Install Nginx

Install the Nginx package.

```
sudo apt install nginx -y
```

Step 3: Verify Nginx Service

Check whether the Nginx service is running.

```
sudo systemctl status nginx
```

You should see that the service is **active (running)**.

Step 4: Verify from the Browser

Open your web browser and navigate to:

```
http://<PUBLIC-IP>
```

Replace <PUBLIC-IP> with the public IP address of your EC2 instance.

If the installation is successful, you should see the **default Nginx Welcome Page**.

Lab Outcome

Nginx installed successfully

Nginx service is running

Successfully accessed the Nginx default page using the EC2 Public IP

Summary

By the end of these notes, you should understand:

- Jenkins fundamentals
 - CI/CD concepts
 - Jenkins Pipelines
 - Jenkins Plugins
 - Distributed Builds
 - Build Triggers
 - Cloud Service Models
 - Prompt Engineering Basics
 - AI Models
 - AWS EC2 Basics
 - SSH into Linux
 - Nginx Installation
-